

GMPO: 组过滤策略优化

导论

GFPO 的一次训练过程可以概括为:

1. 从训练集中采样问题 q
2. 旧策略 $\pi_{\theta_{\text{old}}}$ 对同一道题生成较大的 G 条 rollout
3. 验证器计算每条回答的奖励 R_i , 并记录回答长度 $|o_i|$
4. 按照目标指标为回答排序。例如按长度 $M_i = -|o_i|$ 或按词元效率 $M_i = \frac{R_i}{|o_i|}$
5. 从 G 条回答中保留排名最高的 k 条, 形成集合 $\mathcal{S}$
6. 仅在保留的 k 条回答中计算奖励均值、标准差和组相对优势:

$$\hat{A}_i = \frac{R_i - \mu_{\mathcal{S}}}{\sigma_{\mathcal{S}} + \varepsilon_{\text{num}}}$$

未被保留的回答设置: $\hat{A}_i = 0, o_i \notin \mathcal{S}$

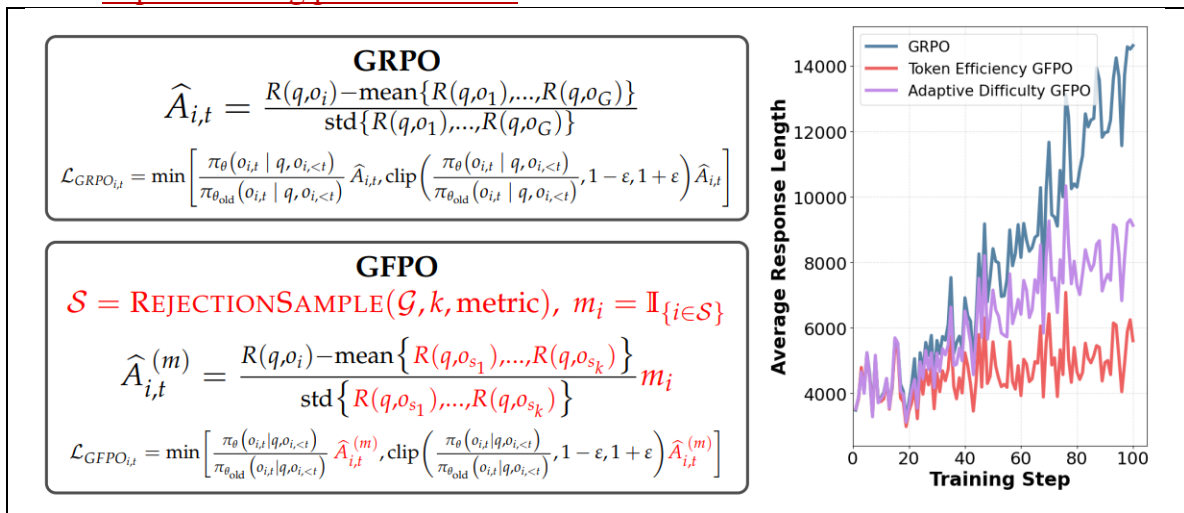
7. 计算保留回答中每个词元的新旧策略概率比 $\rho_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}$
8. 使用 GRPO 式裁剪目标计算损失
9. 反向传播并更新模型参数 θ
10. 更新后的策略进入下一轮 rollout

即每道题多生成 G 条回答, 按长度或奖励效率筛选出 k 条, 只利用保留回答的相对优势更新策略。

GFPO 在 GRPO 基础上提出的一种方法, 目标是缓解推理模型在强化学习过程中出现的**回答长度膨胀**: 同一道题先生成更多候选回答, 再按照长度、单位词元奖励等指标过滤, 只使用保留下来的回答计算相对优势和策略梯度。

GFPO 修改哪些回答能够参与策略更新, 论文将这种过滤视为一种隐式奖励塑形: 正确性仍由原奖励函数衡量, 而简洁性等额外属性通过筛选过程控制。

(<https://arxiv.org/pdf/2508.09726>)



GRPO 对同一道题生成 G 条回答 $\mathcal{G} = \{o_1, o_2, \dots, o_G\}$ ，然后根据奖励计算组内相对优势 $\hat{A}_i = \frac{R_i - \mu_R}{\sigma_R + \varepsilon_{\text{num}}}$

问题是，强化学习可能逐渐鼓励模型生成越来越长的推理过程。更长的回答有时确实有助于解决难题，但也可能包含：重复推导；无效试探；已经得到答案后仍继续推理；大量不增加正确率的填充词元。

GFPO 论文指出，仅仅在奖励中加入长度惩罚，仍可能不足以阻止这种长度膨胀。因此，它直接在训练数据层面筛选较短或更高效的回答。

对于一道问题 q ，GFPO 分为四步。

1. 普通 GRPO 可能生成 $G=8$ 条回答；GFPO 会增加采样数量，例如 $G=16$ ，从而提高候选集中出现“正确且简洁”回答的概率。

2. 论文主要研究两种筛选指标

回答长度

令第 i 条回答的长度为： $T_i = |o_i|$

若目标是简洁，就按长度从小到大排序，选择最短的 k 条回答：

$$S = \text{TopK}_{\text{short}}\{o_1, \dots, o_G\}$$

这里 S 是保留集合，并且 $|S| = k < G$

词元效率

定义第 i 条回答的词元效率： $E_i = \frac{R_i}{T_i}$ ，其中 R_i 是回答奖励； T_i 是回答长度；

E_i 表示平均每个词元产生多少奖励。然后保留词元效率最高的 k 条回答：

$$S = \text{TopK}\left\{\frac{R_1}{T_1}, \dots, \frac{R_G}{T_G}\right\}$$

长度筛选强调“尽可能短”，词元效率筛选强调长回答并非一定不好，但它必须用更高的奖励证明额外词元是有价值的。

3. 定义二值掩码： $m_i = \begin{cases} 1, & i \in S, \\ 0, & i \notin S. \end{cases}$ ，只使用保留回答的奖励计算均值

$$\mu_S = \frac{1}{k} \sum_{j \in S} R_j, \text{ 以及标准差 } \sigma_S = \sqrt{\frac{1}{k} \sum_{j \in S} (R_j - \mu_S)^2}$$

$$\text{GFPO 的优势为 } \hat{A}_i^{(m)} = \frac{R_i - \mu_S}{\sigma_S + \varepsilon_{\text{num}}} m_i$$

因此被保留回答： $m_i = 1$ ，获得正常优势；被过滤回答： $m_i = 0$ ，优势直接变成 0。也就是说 $i \notin S \implies \hat{A}_i^{(m)} = 0$ ，被过滤回答不会产生策略梯度。

4. 定义词元概率比 $\rho_{i,t}(\theta) = \frac{\pi_{\theta}(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})}$

GFPO 的裁剪目标可以写成：

$$\mathcal{J}_{\text{GFPO}}(\theta) = \mathbb{E} \left[\frac{1}{\sum_{i=1}^G T_i} \sum_{i=1}^G \sum_{t=1}^{T_i} \min(\rho_{i,t}(\theta) \hat{A}_i^{(m)}, \text{clip}(\rho_{i,t}(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_i^{(m)}) - \beta D_{\text{KL}} + \gamma \mathcal{H} \right]$$

其中 $T_i = |o_i|$ 是回答长度； βD_{KL} 是 KL 惩罚； $\gamma \mathcal{H}$ 是熵奖励；论文实验使用了类似 DAPO 的全词元损失聚合方式。

GFPO 的主要改动集中在 $\hat{A}_i \rightarrow \hat{A}_i^{(m)}$ ，即先筛选回答，再计算优势。论文也因此指出，GFPO 可以与 DAPO、Dr. GRPO 等不同的组策略优化变体组合。

举例：

假设同一道数学题生成六条回答：G=6，每条回答的长度和奖励如下：

回答	长度 T_i	奖励 R_i
o_1	90	1.0
o_2	120	0
o_3	150	0.8
o_4	220	1.0
o_5	300	0
o_6	400	1.0

现在使用**长度过滤**，保留最短的三条 $k=3$ ，因此 $S = \{o_1, o_2, o_3\}$ ，对应掩码为 $\mathbf{m} = [1, 1, 1, 0, 0, 0]$

计算保留组的平均奖励： $\mu_S = \frac{1.0 + 0 + 0.8}{3} = 0.6$

计算其的标准差 $\sigma_S = \sqrt{\frac{(1.0 - 0.6)^2 + (0 - 0.6)^2 + (0.8 - 0.6)^2}{3}} = 0.432$

第一条回答优势 $\hat{A}_1^{(m)} = \frac{1.0 - 0.6}{0.432} \times 1 \approx 0.926$

第二条回答优势 $\hat{A}_2^{(m)} = \frac{0 - 0.6}{0.432} \times 1 \approx -1.389$

第三条回答优势 $\hat{A}_3^{(m)} = \frac{0.8 - 0.6}{0.432} \times 1 \approx 0.463$

后三条回答未被选中 $\hat{A}_4^{(m)} = \hat{A}_5^{(m)} = \hat{A}_6^{(m)} = 0$

因此最终优势为 $\hat{\mathbf{A}}^{(m)} = [0.926, -1.389, 0.463, 0, 0, 0]$

对于 o_1 ： $\hat{A}_1^{(m)} = 0.926 > 0$ ，它既短又获得高奖励，因此模型会提高它的生成概率；对于 o_2 ： $\hat{A}_2^{(m)} = -1.389 < 0$ ，它虽然很短，但答案错误，因此模型会降低它的

生成概率；对于 o_3 ： $\hat{A}_3^{(m)} = 0.463 > 0$ ，它比较短，奖励也高于保留组平均值，因此会被适度强化；对于 o_4 ： $\hat{A}_4^{(m)} = 0$ ，尽管它的奖励为 1，但它没有通过长度过滤，因此不参与本次策略更新。

这体现 GFPO 的机制：过滤指标决定哪些回答有资格学习；奖励决定保留回答中哪些应该增强或削弱，所以 GFPO 并不是简单地奖励短回答。一个短但错误的回答仍会得到负优势。

假设 o_1 中某个词元的旧策略概率为： $\pi_{\theta_{old}} = 0.20$ ；新策略概率为 $\pi_{\theta} = 0.24$ ，那么概率比为 $\rho_{1,t} = \frac{0.24}{0.20} = 1.20$

已知 $\hat{A}_1^{(m)} = 0.926$ ，未裁剪贡献为 $\rho_{1,t} \hat{A}_1^{(m)} = 1.20 \times 0.926 = 1.1112$ ，若裁剪区间为 $[0.8, 1.2]$ ，则该词元位于上边界，最终贡献仍为 1.1112

而对于被过滤的 o_4 ，由于 $\hat{A}_4^{(m)} = 0$ ，无论其词元概率比是多少 $\rho_{4,t} \hat{A}_4^{(m)} = 0$ ，所以它不会产生策略梯度。

为什么不直接把长度写进奖励？

一种直接方法是构造 $R_i^{total} = R_i^{correct} - \lambda_{len} T_i$ ，但这会带来几个问题。

首先，正确性与长度被压缩成一个标量，需要仔细调整 λ_{len} ，如果惩罚太弱，模型仍会不断变长；如果惩罚太强，模型可能为了缩短回答而牺牲正确率。

其次，不同难度问题需要的合理推理长度不同。固定长度惩罚很难同时适应简单题和复杂题。

GFPO 将两个作用分开：利用筛选指标控制简洁性或其他目标属性；利用奖励函数判断回答质量和正确性。

因此不需要把所有目标硬塞进同一个奖励标量。论文把这种筛选称为隐式、灵活的奖励塑形。

Adaptive Difficulty GFPO

固定保留数量 k 对所有问题一视同仁，但不同问题的难度不同。

对于简单题，模型可能生成很多正确答案，此时可以进行更激进的过滤，只保留少量最简洁回答。

对于困难题，正确路径本来就很稀少。如果保留数量过少，可能把有价值的探索轨迹过滤掉。

GFPO 因此提出 **Adaptive Difficulty GFPO**。

对一道问题的 G 条回答计算平均奖励 $\bar{R}_q = \frac{1}{G} \sum_{i=1}^G R_i$ ，如果 $\bar{R}_q$ 较高，说明大多数

回答表现不错，问题相对简单。如果 $\bar{R}_q$ 较低，说明模型经常回答错误，问题相对困难。

论文维护历史问题平均奖励的分位数，并将问题划分为简单、中等、困难和非常困难。论文实验在每题采样 16 条回答时采用简单题保留 4 条；中等题保留 6 条；困难和非常困难题保留 8 条。

因此简单问题：小 k ，强过滤，重点压缩长度；困难问题：大 k ，多保留，重点维持探索和正确率。

这种方法把训练资源更多分配给困难问题，同时对简单问题进行更激进的简洁性优化。

GFPO 的主要代价是增加训练时采样成本。例如 GRPO：每题生成 8 条，8 条参与训练；GFPO：每题生成 16 条，只保留 8 条参与训练。GFPO 实际进行了更多 rollout，但只有筛选后的部分产生梯度。

它采用的是一种训练与推理之间的成本交换训练时多采样，学习更简洁的推理方式，部署时生成更少词元。

论文在 Phi-4-reasoning 上报告，GFPO 在保持准确率的同时，将 GRPO 的长度膨胀削减约 46% 至 71%；按词元效率筛选时，削减范围进一步达到约 71% 至 85%。

GFPO 的局限性

1. 过滤指标可能产生偏差。只保留最短回答可能使模型偏好省略必要推导，尤其是在需要完整证明或可解释过程的任务中。
2. 增加 G 会提高 rollout 和验证成本。节省的是长期推理成本，但前期训练更昂贵。（**Rollout** 可以理解为让当前策略模型从一个初始问题开始，实际执行一次完整的生成过程，并记录它产生的整条轨迹。在大语言模型强化学习中，rollout 通常就是“输入问题 → 模型逐词生成回答 → 得到完整回答 → 计算奖励”）
3. G 和 k 需要调节。若 k 太小，可能丢失探索性轨迹；若 k 太大，过滤作用又会变弱。
4. 筛选无法替代正确性奖励。短回答只获得参与比较的资格，最终仍需要可靠的验证器判断答案质量。
5. 词元效率依赖奖励尺度。若奖励函数本身存在漏洞： $E_i = \frac{R_i}{T_i}$ ，也可能放大奖励设计中的问题。

总结

GFPO 的思想可以概括为多生成，精筛选，少学习；用更多训练时探索，换取更短的推理时回答。